2024-SC2-COS10033-Advanced Programming

Assignment 1 Report

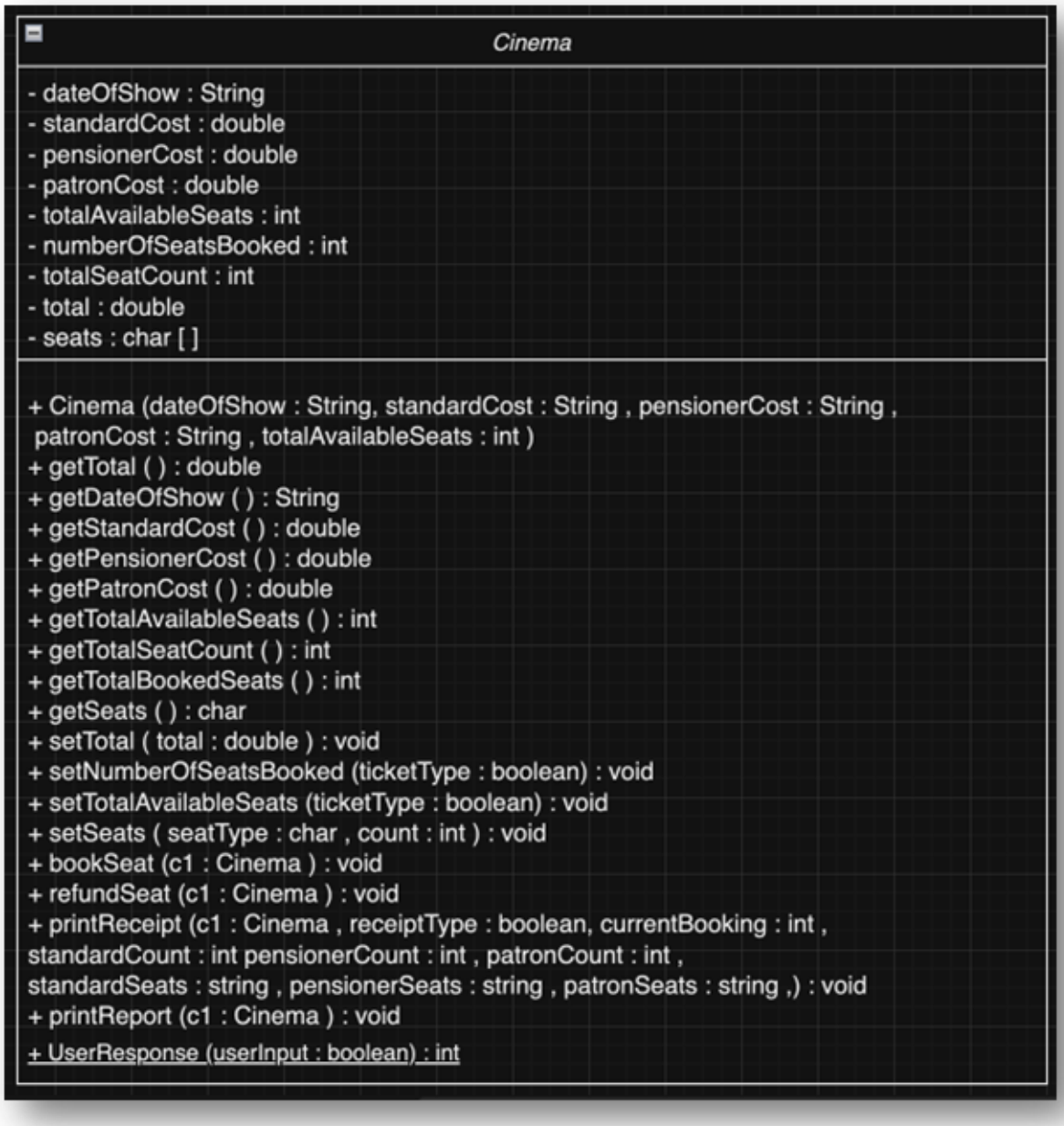
Name: Banuka Jayathilaka ***Student ID: 105200902***

Lab class: Wednesday ***Teacher: MS LUSHAKA NISANSALA***

Due Date: 11 Aug at 23:59

Date Submitted: 11 Aug at 03.30

UML Diagram


Cinema
- dateOfShow : String
- standardCost : double
- pensionerCost : double
- patronCost : double
- totalAvailableSeats : int
- numberOfSeatsBooked : int
- totalSeatCount : int
- total : double
- seats : char [ ]
+ Cinema (dateOfShow : String, standardCost : String , pensionerCost : String ,
patronCost : String , totalAvailableSeats : int)
+ getTotal ( ) : double
+ getDateOfShow ( ) : String
+ getStandardCost ( ) : double
+ getPensionerCost ( ) : double
+ getPatronCost ( ) : double
+ getTotalAvailableSeats ( ) : int
+ getTotalSeatCount ( ) : int
+ getTotalBookedSeats ( ) : int
+ getSeats ( ) : char
+ setTotal (total : double) : void
+ setNumberOfSeatsBooked (ticketType : boolean) : void
+ setTotalAvailableSeats (ticketType : boolean) : void
+ setSeats (seatType : char , count : int) : void
+ bookSeat (c1 : Cinema) : void
+ refundSeat (c1 : Cinema) : void
+ printReceipt (c1 : Cinema , receiptType : boolean, currentBooking : int ,
standardCount : int pensionerCount : int , patronCount : int ,
standardSeats : string , pensionerSeats : string , patronSeats : string ,) : void
+ printReport (c1 : Cinema) : void
+ UserResponse (userInput : boolean) : int

Cinema.Java

Source code

```
/*Name- Banuka Jayathilaka
Student ID- 105200902
Stage A- Cinema.java*/

package com.Stage.code;

import java.util.Scanner;

public class Cinema {

// Private instance variables for the class
private String dateOfShow; // Date on which the show will take place
private double standardCost; // Cost for a standard ticket
private double pensionerCost; // Cost for a pensioner ticket
private double patronCost; // Cost for a frequent patron ticket
private int totalAvailableSeats; // Total number of seats currently available for
booking
private int numberOfSeatsBooked; // Total number of seats that have been booked so
far
private int totalSeatCount; // Total number of seats in the cinema
private double total; // Accumulated total revenue generated from bookings
private char[] seats; // Array representing the status of each seat (e.g., booked or
available)

// Parameterized constructor for initializing the Cinema object
public Cinema(String dateOfShow, double standardCost, double pensionerCost, double
patronCost,
int totalAvailableSeats) {
// Assign the constructor parameters to the corresponding instance variables
this.dateOfShow = dateOfShow;
this.standardCost = standardCost;
this.pensionerCost = pensionerCost;
this.patronCost = patronCost;
this.totalAvailableSeats = totalAvailableSeats;

// Initialize instance variables with default values
numberOfSeatsBooked = 0; // Initially, no seats are booked
totalSeatCount = totalAvailableSeats; // Set total seat count to the initially
provided number
total = 0.0; // Total revenue starts at zero

// Initialize the seats array to represent the status of all seats
seats = new char[totalSeatCount];

// Mark all seats as available ('-') at the start
for (int i = 0; i < totalAvailableSeats; i++) {
seats[i] = '-';
}
}
```

```java
// Getter and Setter methods for accessing and modifying the private variables

// Returns the total revenue generated from bookings
public double getTotal() {
return total;
}

// Updates the total revenue based on the ticket type and action (booking or
// refund)
public void setTotal(double total, boolean ticketType) {
if (ticketType) // If true, a seat is being booked, so the total increases
this.total += total;
else // If false, a seat is being refunded, so the total decreases
this.total -= total;
}

// Returns the date of the show
public String getDateOfShow() {
return dateOfShow;
}

// Returns the cost of a standard ticket
public double getStandardCost() {
return standardCost;
}

// Returns the cost of a pensioner ticket
public double getPensionerCost() {
return pensionerCost;
}

// Returns the cost of a patron ticket
public double getPatronCost() {
return patronCost;
}

// Returns the number of seats currently available for booking
public int getTotalAvailableSeats() {
return totalAvailableSeats;
}

// Adjusts the count of available seats based on whether a booking or refund
// occurred
public void setTotalAvailableSeats(boolean ticketType) {
if (ticketType) // If true, a seat is being refunded, so the available seat count
increases
totalAvailableSeats++;
else // If false, a seat is being booked, so the available seat count decreases
totalAvailableSeats--;
}

// Returns the number of seats that have been booked so far
```

```
public int getNumberOfSeatsBooked() {
return numberOfSeatsBooked;
}

// Adjusts the count of booked seats based on whether a booking or refund
// occurred
public void setNumberOfSeatsBooked(boolean ticketType) {
if (ticketType) // If true, a seat is being booked, so the booked seat count
increases
numberOfSeatsBooked++;
else // If false, a seat is being refunded, so the booked seat count decreases
numberOfSeatsBooked--;
}

// Returns the total number of seats in the cinema
public int getTotalSeatCount() {
return totalSeatCount;
}

// Returns the status of a specific seat (e.g., booked or available)
public char getSeats(int count) {
return seats[count]; // Retrieves the seat status at the given index
}

// Updates the status of a specific seat (e.g., to booked or available)
public void setSeats(char seatType, int count) {
this.seats[count] = seatType; // Sets the seat status at the given index
}

// Method to handle the booking of seats
public void bookSeat(Cinema c1) {

// Local variables for tracking booking details
int standardCount = 0, pensionerCount = 0, patronCount = 0; // Counts of each ticket
type booked
String standardSeats = " ", pensionerSeats = " ", patronSeats = " "; // Lists of seat
numbers for each ticket
// type
int currentBooking = 0; // Tracks the number of seats booked in the current
transaction
int seatNumber = 0; // Holds the seat number input by the user
int seatType = 0; // Holds the type of seat (standard, pensioner, patron) input by
the user

// Prompting the user for seat numbers to book
System.out.println(" ");
System.out.println("Enter your desired Seat Numbers continuously. Enter -1 to quit");

// Loop to process seat bookings until the user quits
while (seatNumber != -1) {

// Check if all seats are already booked
if (c1.getTotalAvailableSeats() <= 0) {
```

```
System.out.println("All seats are booked !");
break; // Exit the loop if no seats are available
}

// Get the seat number from the user
seatNumber = Demo.UserResponse(true);

// Validate the seat number input by the user
if (seatNumber > getTotalSeatCount() || seatNumber < 1) {
if (seatNumber != -1) { // Exclude the quit command (-1) from validation
System.out.println("Seat Number you entered doesn't exist");
continue; // Skip the current iteration if the input is invalid
}
}
System.out.println("");

// If the user is done booking, print the receipt and exit the loop
if (seatNumber == -1) {
boolean receiptType = true; // Indicates that this is a booking receipt
printReceipt(c1, receiptType, currentBooking, standardCount, pensionerCount,
patronCount, standardSeats,
pensionerSeats, patronSeats);
break;
}

// Check if the seat is already booked
if (c1.getSeats(seatNumber - 1) != '-') {
System.out.println("Sorry, that Seat Number is already booked !");
continue; // Skip the current iteration if the seat is already booked
}

// Get the seat type from the user
seatType = Demo.UserResponse(false);

// Validate the seat type input by the user
if (seatType > 3 || seatNumber < 1) {
System.out.println("Seat Type you entered doesn't exist");
continue; // Skip the current iteration if the input is invalid
}
System.out.println("");

// Update the seat array and booking details based on the user's input
switch (seatType) {
case 1:
standardSeats += String.valueOf(seatNumber) + ","; // Append the seat number to the
standard seats list
c1.setSeats('S', seatNumber - 1); // Mark the seat as booked with 'S' for standard
c1.setTotal(c1.getStandardCost(), true); // Update the total revenue
standardCount++; // Increment the count of standard seats booked
break;

case 2:
```

```
pensionerSeats += String.valueOf(seatNumber) + ","; // Append the seat number to the
pensioner seats
// list
c1.setSeats('P', seatNumber - 1); // Mark the seat as booked with 'P' for pensioner
c1.setTotal(c1.getPensionerCost(), true); // Update the total revenue
pensionerCount++; // Increment the count of pensioner seats booked
break;

case 3:
patronSeats += String.valueOf(seatNumber) + ","; // Append the seat number to the
patron seats list
c1.setSeats('F', seatNumber - 1); // Mark the seat as booked with 'F' for patron
c1.setTotal(c1.getPatronCost(), true); // Update the total revenue
patronCount++; // Increment the count of patron seats booked
break;
}

currentBooking++; // Increment the count of seats booked in this transaction
c1.setNumberOfSeatsBooked(true); // Update the total number of seats booked
c1.setTotalAvailableSeats(false); // Update the total number of available seats
System.out.println("Booked Successfully");
}
}

// Method to handle the refunding of seats
public void refundSeat(Cinema c1) {

// Local variables for tracking refund details
int standardCount = 0, pensionerCount = 0, patronCount = 0; // Counts of each ticket
type refunded
String standardSeats = " ", pensionerSeats = " ", patronSeats = " "; // Lists of seat
numbers for each ticket
// type refunded
int currentRefund = 0; // Tracks the number of seats refunded in the current
transaction
int seatNumber = 0; // Holds the seat number input by the user
char seatType = ' '; // Holds the type of seat (S, P, F) being refunded

// Prompting the user for seat numbers to refund
System.out.println(" ");
System.out.println("Enter your booked Seat Numbers continuously to Refund. Enter -1
to quit");

// Loop to process seat refunds until the user quits
while (seatNumber != -1) {

// Check if any seats are booked
if (c1.getNumberOfSeatsBooked() <= 0) {
System.out.println("No Seats are booked to Refund");
break; // Exit the loop if no seats are booked
}

// Get the seat number from the user
```

```
seatNumber = Demo.UserResponse(true);

// Validate the seat number input by the user
if (seatNumber > getTotalSeatCount() || seatNumber < 1) {
if (seatNumber != -1) { // Exclude the quit command (-1) from validation
System.out.println("Seat Number you entered doesn't exist");
continue; // Skip the current iteration if the input is invalid
}
}

// If the user is done refunding, print the receipt and exit the loop
if (seatNumber == -1) {
boolean receiptType = false; // Indicates that this is a refund receipt
printReceipt(c1, receiptType, currentRefund, standardCount, pensionerCount,
patronCount, standardSeats,
pensionerSeats, patronSeats);
break;
}

// Check if the seat is already refunded
if (c1.getSeats(seatNumber - 1) == '-') {
System.out.println("Sorry, that Seat Number is not booked yet !");
continue; // Skip the current iteration if the seat is not booked
}

seatType = c1.getSeats(seatNumber - 1); // Retrieve the seat type from the array

// Update the seat array and refund details based on the seat type
switch (seatType) {
case 'S':
standardSeats += String.valueOf(seatNumber) + ","; // Append the seat number to the
standard seats list
c1.setSeats('-', seatNumber - 1); // Mark the seat as available with '-' for standard
c1.setTotal(c1.getStandardCost(), false); // Update the total revenue
standardCount++; // Increment the count of standard seats refunded
break;

case 'P':
pensionerSeats += String.valueOf(seatNumber) + ","; // Append the seat number to the
pensioner seats
// list
c1.setSeats('-', seatNumber - 1); // Mark the seat as available with '-' for
pensioner
c1.setTotal(c1.getPensionerCost(), false); // Update the total revenue
pensionerCount++; // Increment the count of pensioner seats refunded
break;

case 'F':
patronSeats += String.valueOf(seatNumber) + ","; // Append the seat number to the
patron seats list
c1.setSeats('-', seatNumber - 1); // Mark the seat as available with '-' for patron
c1.setTotal(c1.getPatronCost(), false); // Update the total revenue
patronCount++; // Increment the count of patron seats refunded
break;
```

```
}

currentRefund++; // Increment the count of seats refunded in this transaction
c1.setNumberOfSeatsBooked(false); // Update the total number of seats booked
c1.setTotalAvailableSeats(true); // Update the total number of available seats
System.out.println("Refunded Successfully");
}
}

// Method to print receipts for bookings and refunds
public void printReceipt(Cinema c1, boolean receiptType, int currentTransaction, int
standardCount,
int pensionerCount, int patronCount, String standardSeats, String pensionerSeats,
String patronSeats) {

// Print the header for the receipt
if (receiptType) // If true, this is a booking receipt
System.out.println("------Booking Receipt------");
else // If false, this is a refund receipt
System.out.println("------Refund Receipt------");

// Print the receipt details
System.out.println("Booking Date: " + c1.getDateOfShow());
System.out.println("Total Tickets: " + currentTransaction);

// Print the details for each type of ticket booked or refunded
if (standardCount > 0) {
System.out.println("Standard Tickets: " + standardCount);
System.out.println("Seats: " + standardSeats);
}
if (pensionerCount > 0) {
System.out.println("Pensioner Tickets: " + pensionerCount);
System.out.println("Seats: " + pensionerSeats);
}
if (patronCount > 0) {
System.out.println("Patron Tickets: " + patronCount);
System.out.println("Seats: " + patronSeats);
}

// Print the total revenue for the transaction
System.out.println("Total Cost: $" + c1.getTotal());
System.out.println("---------------------------");
}

public void printReport(Cinema c1) {
System.out.println();

}
}
```

Demo.java

Source code

```
/*Name- Banuka Jayathilaka
  Student ID- 105200902
  Stage A- Demo.java*/

package com.Stage.code;

import java.util.Scanner;

public class Demo {

    // This method is responsible for collecting user input.
    // Depending on the boolean argument 'userInput', it will either request a seat
number or a seat type from the user.
    @SuppressWarnings("resource")
    public static int UserResponse(boolean userInput) {
        Scanner sc = new Scanner(System.in);
        if (userInput) {
            // When userInput is true, prompt the user to enter a seat number.
            System.out.println(" ");
            System.out.print("Seat Number: ");
            return sc.nextInt();
        } else if (!userInput) {
            // When userInput is false, present the user with a list of seat types
and prompt for selection.
            System.out.println("Choose your desired Seat Type and Enter its
corresponding number");
            System.out.println("1. Standard");
            System.out.println("2. Pensioner");
            System.out.println("3. Frequent Patron");
            System.out.println(" ");
            System.out.print("Seat Type: ");
            return sc.nextInt();
        } else {
            // Close the scanner if neither condition is met (this line is not
expected to execute due to the logic flow).
            sc.close();
            return 0;
        }
    }

    public static void main(String[] args) {

        // Declaration and initialization of variables that will hold user inputs
such as row count and menu choice.
        int rowCount;
        int choice = 0;

        // Create a new scanner object for capturing input from the user.
        Scanner sc = new Scanner(System.in);
```

```
        // Introduction section for the Melbourne Cinema Manager Interface, where
initial configuration inputs are requested from the manager.
        System.out.println(" Melbourne Cinema Manager Interface");
        System.out.println("----------------------------------------");
        System.out.println(" ");

        // Asking the user to input the number of rows in the cinema, which will be
used to calculate the total number of seats.
        System.out.print("Enter row count : ");
        rowCount = sc.nextInt();
        System.out.println(" ");
        sc.nextLine(); // Consuming the newline character to avoid input issues
later.

        // Requesting the manager to input the date of the show.
        System.out.print("Enter date of the show : ");
        String tempDate = sc.nextLine();
        System.out.println(" ");

        // Asking for the cost of a standard ticket.
        System.out.print("Enter price of a standard ticket : ");
        double tempSCost = sc.nextDouble();
        System.out.println(" ");

        // Asking for the cost of a pensioner ticket.
        System.out.print("Enter price of a pensioner ticket : ");
        double tempPCost = sc.nextDouble();
        System.out.println(" ");

        // Asking for the cost of a frequent patron ticket.
        System.out.print("Enter price of a frequent patron ticket : ");
        double tempFCost = sc.nextDouble();
        System.out.println(" ");

        // Welcome message after setting up the cinema details.
        System.out.println("----WELCOME TO MELBOURNE CINEMA----");
        System.out.println(" ");

        // Creating an instance of the Cinema class, initializing it with the date of
the show, ticket prices, and the total seat count (calculated by multiplying row
count by 10).
        Cinema c1 = new Cinema(tempDate, tempSCost, tempPCost, tempFCost, rowCount *
10);

        // This loop will continue to run until the user selects option 5 to exit.
        while (choice != 5) {

            // Displaying the main menu options to the user.
            System.out.println("----------------------------------");
            System.out.println(" ");
            System.out.println("1. Display available seats");
            System.out.println("2. Book a seat");
            System.out.println("3. Refund a seat");
            System.out.println("4. Display a report");
            System.out.println("5. Exit");
```

```
            System.out.println(" ");
            System.out.print("Enter choice : ");
            choice = sc.nextInt(); // Capturing the user's menu choice.

            switch (choice) {
                case 1:
                    // If the user chooses to display available seats, the seats are
printed in a structured manner.
                    System.out.println(" ");
                    System.out.println(" SCREEN");
                    System.out.println("-------------------------------------------
-----------------");

                    System.out.print(" ");

                    // Looping through the seats to display them in rows. Each row
contains 10 seats with a space after every 5 seats to represent the aisle.
                    for (int i = 0; i < c1.getTotalSeatCount(); i++) {
                        if (i % 5 == 0 && i != 0) // Adding a space to indicate the
aisle after every 5 seats.
                            System.out.print(" ");
                        if (i % 10 == 0 && i != 0) // Moving to the next row after
every 10 seats.
                            System.out.println();
                        if (i < 10) {
                            // For seat numbers less than 10, extra space is added to
align the display for better readability.
                            System.out.print((i + 1) + ":" + c1.getSeats(i) + " ");
                            continue;
                        }
                        System.out.print((i + 1) + ":" + c1.getSeats(i) + " ");
                    }

                    System.out.println(" ");
                    System.out.println(" ");

                    // Display the total number of available seats after the layout
is printed.
                    System.out.println("Number of Available Seats : " +
c1.getTotalAvailableSeats());
                    System.out.println(" ");

                    break;

                case 2:
                    // If the user selects the option to book a seat, the booking
process is initiated.
                    c1.bookSeat(c1);
                    break;

                case 3:
                    // If the user selects the option to refund a seat, the refund
process is initiated.
                    c1.refundSeat(c1);
                    break;
```

```java
                case 4:
                    // If the user chooses to display a report, the report is
generated and shown.
                    c1.printReport(c1);
                    break;

                case 5:
                    // If the user selects the option to exit, a termination message
is displayed, resources are closed, and the program is terminated.
                    System.out.println(" ");
                    System.out.println("Program Terminated");
                    sc.close();
                    System.exit(choice = 5);
            }
        }
    }
}
```

Manager user interface

```
Console
Demo [Java Application] C:\Program Files\Java\jdk-22\bin\javaw.exe  (5 Aug 2024, 17:54:55) [pid: 18280]
  Melbourne Cinema Manager Interface
-------------------------------------

Enter row count : 3

Enter date of the show : 07/08/2024

Enter price of a standard : 100

Enter price of a pensioner : 200

Enter price of a frequent : 500

----WELCOME TO MELBOURNE CINEMA----

---------------------------------

1. Displaying available seats
2. Booking a seat
3. Refunding a seat
4. Displaying a report
5. Exit

Enter choice : 1
```

Displaying available seats and total available seats at start

```
Console
Demo [Java Application] C:\Program Files\Java\jdk-22\bin\javaw.exe  (5 Aug 2024, 17:54:55) [pid: 18280]

Enter row count : 3

Enter date of the show : 07/08/2024

Enter price of a standard : 100

Enter price of a pensioner : 200

Enter price of a frequent : 500

----WELCOME TO MELBOURNE CINEMA----

---------------------------------

1. Displaying available seats
2. Booking a seat
3. Refunding a seat
4. Displaying a report
5. Exit

Enter choice : 1

                        SCREEN
-------------------------------------------------------------
 1:-   2:-   3:-   4:-   5:-     6:-   7:-   8:-   9:-   10:-
11:-  12:-  13:-  14:-  15:-    16:-  17:-  18:-  19:-  20:-
21:-  22:-  23:-  24:-  25:-    26:-  27:-  28:-  29:-  30:-

Number of Available Seats : 30

---------------------------------

1. Displaying available seats
2. Booking a seat
3. Refunding a seat
4. Displaying a report
```

Booking a Seat and a receipt being generated

```
Console
Demo [Java Application] C:\Program Files\Java\jdk-22\bin\javaw.exe  (5 Aug 2024, 18:03:39) [pid: 5452]

1. Displaying available seats
2. Booking a seat
3. Refunding a seat
4. Displaying a report
5. Exit

Enter choice : 2

Enter your desired Seat Numbers continuously. Enter -1 to quit

Seat Number: 2

Choose your desired Seat Types and Enter its relevant number
1. Standard
2. Pensioner
3. Frequent Patron

Seat Type: 1

Booked Successfully

Seat Number: 4

Choose your desired Seat Types and Enter its relevant number
1. Standard
2. Pensioner
3. Frequent Patron

Seat Type: 2

Booked Successfully

Seat Number: -1


          RECEIPT
```

```
Console ×
Demo [Java Application] C:\Program Files\Java\jdk-22\bin\javaw.exe  (5 Aug 2024, 18:03:39) [pid: 5452]

Booked Successfully

Seat Number: 4

Choose your desired Seat Types and Enter its relevant number
1. Standard
2. Pensioner
3. Frequent Patron

Seat Type: 2

Booked Successfully

Seat Number: -1


            RECEIPT
******************************

Date : 07/08/2024
Number of seats booked : 2

1 * Standard @ $100.0 = $100.0 Seat(s): 2
1 * Pensioner @ $200.0 = $200.0 Seat(s): 4
0 * Frequent @ $500.0 = $0.0 Seat(s):

           Total : $ 300.00
---------------------------------

1. Displaying available seats
2. Booking a seat
3. Refunding a seat
4. Displaying a report
5. Exit

Enter choice :
```

Validation checking for bookSeat

1. when a previously reserved seat number is entered when making a new reservation.
2. When a customer attempts to reserve a seat that is non-exiting (for example, seat 40 in this instance),

```
Console ×
Demo [Java Application] C:\Program Files\Java\jdk-22\bin\javaw.exe  (5 Aug 2024, 18:03:39) [pid: 5452]

1. Displaying available seats
2. Booking a seat
3. Refunding a seat
4. Displaying a report
5. Exit

Enter choice : 2

Enter your desired Seat Numbers continuously. Enter -1 to quit

Seat Number: 2

Sorry, that Seat Number is already booked !

Seat Number: 40
Seat Number you entered doesn't exist

Seat Number: 6

Choose your desired Seat Types and Enter its relevant number
1. Standard
2. Pensioner
3. Frequent Patron

Seat Type: 3

Booked Successfully

Seat Number: 8

Choose your desired Seat Types and Enter its relevant number
1. Standard
2. Pensioner
3. Frequent Patron

Seat Type: 2
```

```
Console ×
Demo [Java Application] C:\Program Files\Java\jdk-22\bin\javaw.exe  (5 Aug 2024, 18:03:39) [pid: 5452]

Booked Successfully

Seat Number: 8

Choose your desired Seat Types and Enter its relevant number
1. Standard
2. Pensioner
3. Frequent Patron

Seat Type: 2

Booked Successfully

Seat Number: -1

           RECEIPT
******************************

Date : 07/08/2024
Number of seats booked : 2

0 * Standard @ $100.0 = $0.0 Seat(s):
1 * Pensioner @ $200.0 = $200.0 Seat(s): 8
1 * Frequent @ $500.0 = $500.0 Seat(s): 6

           Total : $ 700.00
--------------------------------

1. Displaying available seats
2. Booking a seat
3. Refunding a seat
4. Displaying a report
5. Exit

Enter choice :
```

Invalid Seat Type message output

```
Seat Number: 7

Choose your desired Seat Types and Enter its relevant number
1. Standard
2. Pensioner
3. Frequent Patron

Seat Type: 5
Seat Type you entered doesn't exist

Seat Number:
```

When all seats are booked

```
Choose your desired Seat Types and Enter its relevant number
1. Standard
2. Pensioner
3. Frequent Patron

Seat Type: 1

Booked Successfully
All seats are booked !
---------------------------------

1. Displaying available seats
2. Booking a seat
3. Refunding a seat
4. Displaying a report
5. Exit

Enter choice :
```

Seat Number 2,4,6,8 have been booked. Display available Seats updated and number of available seats updated

```
---------------------------------

1. Displaying available seats
2. Booking a seat
3. Refunding a seat
4. Displaying a report
5. Exit

Enter choice : 1

                        SCREEN
-------------------------------------------------------------
 1:-   2:S   3:-   4:P   5:-     6:F   7:-   8:P   9:-   10:-
11:-  12:-  13:-  14:-  15:-    16:-  17:-  18:-  19:-  20:-
21:-  22:-  23:-  24:-  25:-    26:-  27:-  28:-  29:-  30:-

Number of Available Seats : 26

---------------------------------

1. Displaying available seats
2. Booking a seat
3. Refunding a seat
4. Displaying a report
5. Exit

Enter choice :
```

Statistical Report after booking

```
----------------------------------

1. Displaying available seats
2. Booking a seat
3. Refunding a seat
4. Displaying a report
5. Exit

Enter choice : 1

                         SCREEN
--------------------------------------------------------------------
 1:-   2:S   3:-   4:P   5:-     6:F   7:-   8:P   9:-   10:-
11:-  12:-  13:-  14:-  15:-    16:-  17:-  18:-  19:-  20:-
21:-  22:-  23:-  24:-  25:-    26:-  27:-  28:-  29:-  30:-

Number of Available Seats : 26

----------------------------------

1. Displaying available seats
2. Booking a seat
3. Refunding a seat
4. Displaying a report
5. Exit

Enter choice : 4

          STATISTICAL REPORT
----------------------------------------
The Number of Seats booked are : 4

The Percentage of the Seats booked are : 13%

The Average Price of Seats booked is : 250.00

----------------------------------

1. Displaying available seats
2. Booking a seat
3. Refunding a seat
4. Displaying a report
5. Exit

Enter choice :
```

Refunding a Seat and a receipt being generated

```
4. Displaying a report
5. Exit

Enter choice : 1

                        SCREEN
-------------------------------------------------------------
 1:-   2:S   3:-   4:P   5:-     6:F   7:-   8:P   9:-   10:-
11:-  12:-  13:-  14:-  15:-    16:-  17:-  18:-  19:-  20:-
21:-  22:-  23:-  24:-  25:-    26:-  27:-  28:-  29:-  30:-

Number of Available Seats : 26

---------------------------------

1. Displaying available seats
2. Booking a seat
3. Refunding a seat
4. Displaying a report
5. Exit

Enter choice : 3

Enter the Seat Numbers you want to refund continuously. Enter -1 to quit

Seat Number: 2

Refunded Successfully

Seat Number: 4

Refunded Successfully

Seat Number: -1


           RECEIPT
******************************

Date : 01/05/2024
Number of seats refunded : 2

1 * Standard @ $100.0 = $100.0 Seat(s): 2
1 * Pensioner @ $200.0 = $200.0 Seat(s): 4
0 * Frequent @ $300.0 = $0.0 Seat(s):

           Total : $ 300.00
---------------------------------

1. Displaying available seats
2. Booking a seat
3. Refunding a seat
4. Displaying a report
5. Exit

Enter choice : 1
```

When available seats are printed, it shows that seat numbers have been successfully refunded and seats are updated

```
           RECEIPT
******************************

Date : 01/05/2024
Number of seats refunded : 2

1 * Standard @ $100.0 = $100.0 Seat(s): 2
1 * Pensioner @ $200.0 = $200.0 Seat(s): 4
0 * Frequent @ $300.0 = $0.0 Seat(s):

           Total : $ 300.00
---------------------------------

1. Displaying available seats
2. Booking a seat
3. Refunding a seat
4. Displaying a report
5. Exit

Enter choice : 1

                        SCREEN
---------------------------------------------------------------
 1:-   2:-   3:-   4:-   5:-     6:F   7:-   8:P   9:-   10:-
11:-  12:-  13:-  14:-  15:-    16:-  17:-  18:-  19:-  20:-
21:-  22:-  23:-  24:-  25:-    26:-  27:-  28:-  29:-  30:-

Number of Available Seats : 28

---------------------------------

1. Displaying available seats
2. Booking a seat
3. Refunding a seat
4. Displaying a report
5. Exit

Enter choice :
```

Validation checking for refundSeat

1. **When a seat number that has been not booked is entered for refunding**
2. **When the user tries to refund a seat that doesn't exit (eg: In this case, seat 40 doesn't exit)**

```
1. Displaying available seats
2. Booking a seat
3. Refunding a seat
4. Displaying a report
5. Exit

Enter choice : 1

                        SCREEN
_____________________________________________________________
 1:-   2:-   3:-   4:-   5:-     6:F   7:-   8:P   9:-   10:-
11:-  12:-  13:-  14:-  15:-    16:-  17:-  18:-  19:-  20:-
21:-  22:-  23:-  24:-  25:-    26:-  27:-  28:-  29:-  30:-

Number of Available Seats : 28

_________________________________

1. Displaying available seats
2. Booking a seat
3. Refunding a seat
4. Displaying a report
5. Exit

Enter choice : 3

Enter the Seat Numbers you want to refund continuously. Enter -1 to quit

Seat Number: 30

Oops! That Seat Number is not booked !

Seat Number: 40
Seat Number you entered doesn't exist

Seat Number: -1
```

Statistical report after refunding

```
1 * Pensioner @ $200.0 = $200.0 Seat(s): 4
0 * Frequent @ $500.0 = $0.0 Seat(s):

           Total : $ 300.00
---------------------------------

1. Displaying available seats
2. Booking a seat
3. Refunding a seat
4. Displaying a report
5. Exit

Enter choice : 1

                        SCREEN
-------------------------------------------------------------
 1:-   2:-   3:-   4:-   5:-     6:F   7:-   8:P   9:-   10:-
11:-  12:-  13:-  14:-  15:-    16:-  17:-  18:-  19:-  20:-
21:-  22:-  23:-  24:-  25:-    26:-  27:-  28:-  29:-  30:-

Number of Available Seats : 28

---------------------------------

1. Displaying available seats
2. Booking a seat
3. Refunding a seat
4. Displaying a report
5. Exit

Enter choice : 4

          STATISTICAL REPORT
---------------------------------------
The Number of Seats booked are : 2

The Percentage of the Seats booked are : 6%

The Average Price of Seats booked is : 350.00

---------------------------------

1. Displaying available seats
2. Booking a seat
3. Refunding a seat
4. Displaying a report
5. Exit

Enter choice :
```

Exiting program

```
--------------------------------

1. Displaying available seats
2. Booking a seat
3. Refunding a seat
4. Displaying a report
5. Exit

Enter choice : 5

Program Terminated
```

JUNIT TESTING

Pass and Fail tests have been done to 4 methods,

- getTotal
- getSeats
- getTotalAvailableSeats
- GetNumberOfSeatsBooked

For these tests to produce the desired results, the bookSeat and refundSeat must function correctly. As a result, the bookSeat and refundSeat features have been evaluated in addition to the functionality of the methods.

CinemaTest.Java

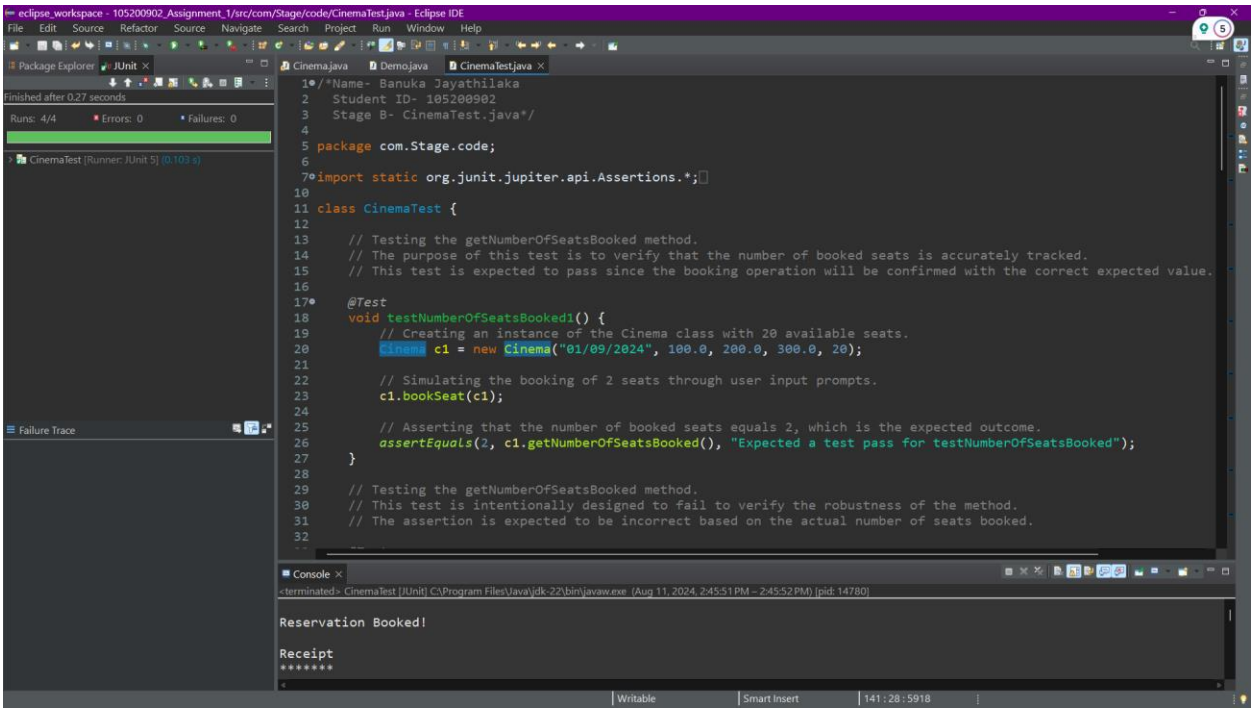
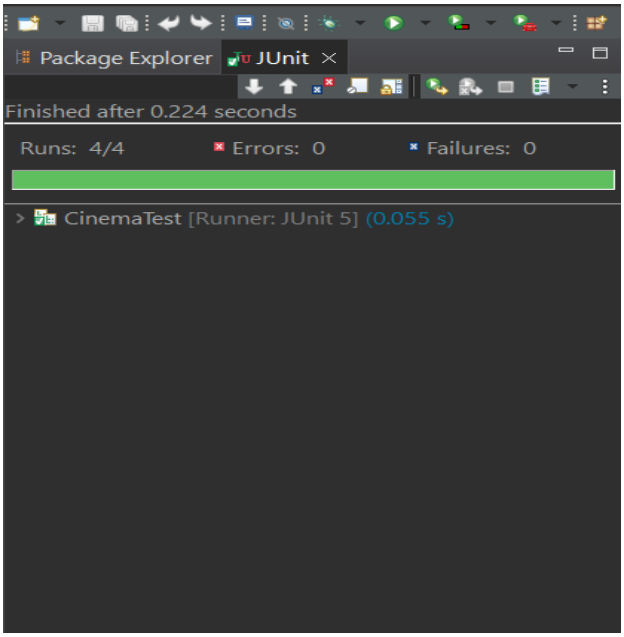
```java
/*Name- Banuka Jayathilaka
Student ID- 105200902
Stage B- CinemaTest.java*/

package com.Stage.code;

import static org.junit.jupiter.api.Assertions.*;

import org.junit.jupiter.api.Test;

class CinemaTest {

// Testing the getNumberOfSeatsBooked method.
// The purpose of this test is to verify that the number of booked seats is
// accurately tracked.
// This test is expected to pass since the booking operation will be confirmed
// with the correct expected value.

@Test
void testNumberOfSeatsBooked1() {
```

```
// Creating an instance of the Cinema class with 20 available seats.
Cinema c1 = new Cinema("01/09/2024", 100.0, 200.0, 300.0, 20);

// Simulating the booking of 2 seats through user input prompts.
c1.bookSeat(c1);

// Asserting that the number of booked seats equals 2, which is the expected
// outcome.
assertEquals(2, c1.getNumberOfSeatsBooked(), "Expected a test pass for
testNumberOfSeatsBooked");
}

// Testing the getNumberOfSeatsBooked method.
// This test is intentionally designed to fail to verify the robustness of the
// method.
// The assertion is expected to be incorrect based on the actual number of seats
// booked.

@Test
void testNumberOfSeatsBooked2() {
// Creating an instance of the Cinema class with 20 available seats.
Cinema c1 = new Cinema("01/09/2024", 100.0, 200.0, 300.0, 20);

// Simulating the booking of only 1 seat through user input prompts.
c1.bookSeat(c1);

// Asserting that the number of booked seats equals 3, which is expected to
// cause the test to fail.
assertEquals(3, c1.getNumberOfSeatsBooked(), "Expected a test fail for
testNumberOfSeatsBooked");
}

// Testing the getTotal method.
// The aim of this test is to check if the total revenue generated from bookings
// is correctly calculated.
// This test is expected to pass as the total amount matches the expected value
// after booking.

@Test
void testGetTotal1() {
// Creating an instance of the Cinema class with 20 available seats and defined
// ticket prices.
Cinema c1 = new Cinema("01/09/2024", 100.0, 200.0, 300.0, 20);

// Simulating the booking of 2 standard seats, which should contribute to the
// total revenue.
```

```
c1.bookSeat(c1);

// Asserting that the total revenue equals 200.0, which corresponds to the cost
// of 2 standard seats.
assertEquals(200.0, c1.getTotal(), "Expected a test pass for testGetTotal");
}

// Testing the getTotal method.
// This test evaluates whether the total revenue is correctly adjusted after a
// refund.
// It is expected to pass, confirming that the refund process is functioning as
// intended.

@Test
void testGetTotal2() {
// Creating an instance of the Cinema class with 20 available seats and defined
// ticket prices.
Cinema c1 = new Cinema("01/09/2024", 100.0, 200.0, 300.0, 20);

// Simulating the booking of 2 seats: one standard and one pensioner.
c1.bookSeat(c1);

// Simulating the refund of one standard seat.
c1.refundSeat(c1);

// Asserting that the total revenue equals 200.0 after the refund is processed.
assertEquals(200.0, c1.getTotal(), "Expected a test pass for testGetTotal");
}

// Testing the getTotal method.
// This test is designed to fail, checking the correctness of the total revenue
// after bookings and refunds.
// The assertion is intentionally incorrect, expecting a different outcome.

@Test
void testGetTotal3() {
// Creating an instance of the Cinema class with 20 available seats and defined
// ticket prices.
Cinema c1 = new Cinema("01/09/2024", 100.0, 200.0, 300.0, 20);

// Simulating the booking of 2 seats: one standard and one pensioner.
c1.bookSeat(c1);

// Simulating the refund of one standard seat.
c1.refundSeat(c1);
```

```
// Asserting that the total revenue equals 100.0, which is expected to be
// incorrect and cause the test to fail.
assertEquals(100.0, c1.getTotal(), "Expected a test fail for testGetTotal");
}

// Testing the getSeats method.
// This test aims to verify that the seats are correctly set to a specific type
// and accurately retrieved.
// The test is expected to pass, confirming that the seat assignment and
// retrieval work as intended.

@Test
void testGetSeats1() {
// Creating an instance of the Cinema class with 30 available seats and defined
// ticket prices.
Cinema c2 = new Cinema("01/09/2024", 200.0, 400.0, 600.0, 30);

// Setting the 10th seat (index 9) to be of type 'S' (Standard).
c2.setSeats('S', 9);

// Asserting that the seat type at index 9 is 'S', which should match the
// expected value.
assertEquals('S', c2.getSeats(9), "Expected a test pass for testGetSeats");
}

// Testing the getSeats method.
// This test checks whether the seat status is correctly updated after booking
// and refund operations.
// The test is expected to pass, confirming that seat statuses are managed
// correctly.

@Test
void testGetSeats2() {
// Creating an instance of the Cinema class with 30 available seats and defined
// ticket prices.
Cinema c2 = new Cinema("01/09/2024", 200.0, 400.0, 600.0, 30);

// Simulating the booking of 3 seats with different types: Pensioner, Patron,
// and Standard.
c2.bookSeat(c2);

// Simulating the refund of one seat, specifically a Patron seat.
c2.refundSeat(c2);
```

```
// Asserting that the seat at index 1 is now available (represented by '-')
// after the refund.
assertEquals('-', c2.getSeats(1), "Expected a test pass for testGetSeat");
}

// Testing the getTotalAvailableSeats method.
// This test verifies that the total count of available seats is correctly
// updated after bookings.
// The test is expected to pass, ensuring that the availability count decreases
// as expected.

@Test
void testGetTotalAvailableSeats1() {
// Creating an instance of the Cinema class with 30 available seats and defined
// ticket prices.
Cinema c2 = new Cinema("01/09/2024", 200.0, 400.0, 600.0, 30);

// Simulating the booking of 3 seats with different types: Pensioner, Patron,
// and Standard.
c2.bookSeat(c2);

// Asserting that the total number of available seats equals 27 after the
// bookings.
assertEquals(27, c2.getTotalAvailableSeats(), "Expected a test pass for
testGetTotalAvailableSeats");
}

// Testing the sgetTotalAvailableSeats method.
// This test is designed to fail by asserting an incorrect value for the total
// available seats after bookings.
// The intention is to verify the test framework's ability to catch errors in
// the seat availability count.

@Test
void testGetTotalAvailableSeats2() {
// Creating an instance of the Cinema class with 30 available seats and defined
// ticket prices.
Cinema c2 = new Cinema("01/09/2024", 200.0, 400.0, 600.0, 30);

// Simulating the booking of 3 seats with different types: Pensioner, Patron,
// and Standard.
c2.bookSeat(c2);

// Asserting that the total number of available seats equals 26, which is
// incorrect and expected to cause the test to fail.
```

```
assertEquals(26, c2.getTotalAvailableSeats(), "Expected a test fail for
testGetTotalAvailableSeats");
}
}
```

OUTPUTS